

UNIT-2

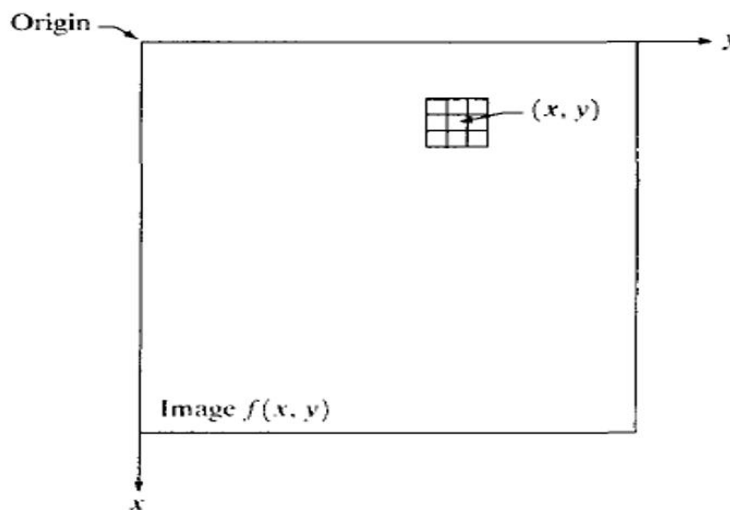
Image Enhancement and filtering in spatial Domain

Image enhancement in the spatial domain refers to techniques that operate directly on the pixels of an image. Unlike the frequency domain, which requires mathematical transformations like the Fourier Transform, spatial domain methods manipulate pixel intensity values within the image's coordinate system to improve visual quality or extract features.

Intensity Transformation Functions: The term **spatial domain** refers to the image plane itself, and image processing methods in this category are based on direct manipulation of pixels in an image. Spatial domain techniques operate directly on the pixels of an image. The spatial domain processes are denoted by the expression

$g(x, y) = T[f(x, y)]$ where $f(x, y)$ is the input image, $g(x, y)$ is the output (processed) image, and T is an operator on f defined over a specified neighborhood about point (x, y) .

The principal approach in defining a neighborhood about a point (x, y) is to use a square or rectangular subimage area centered at (x, y) , as Fig. 3.1 shows. The center of the subimage is moved from pixel to pixel starting, say, at the top left corner. The operator T is applied at each location (x, y) to yield the output, g , at that location. The process utilizes only the pixels in the area of the image spanned by the neighborhood. Although other neighborhood shapes, such as ap-



proximations to a circle, sometimes are used, square and rectangular arrays are by far the most predominant because of their ease of implementation.

The simplest form of T is when the neighborhood is of size 1×1 (that is, a single pixel). In this case, g depends only on the value of f at (x, y) , and T becomes a *gray-level* (also called an *intensity* or *mapping*) *transformation function* of the form

$$s = T(r) \quad (3.1-2)$$

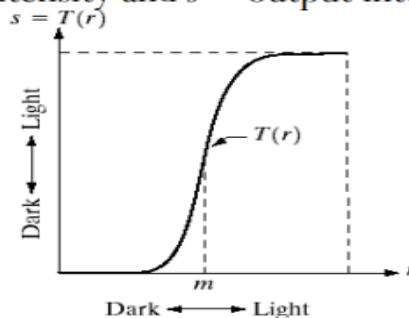
where, for simplicity in notation, r and s are variables denoting, respectively, the gray level of $f(x, y)$ and $g(x, y)$ at any point (x, y) .

Gray Level Transformation

Transforms intensity of an original image into intensity of an output image using a function:

$$s = T(r)$$

where r = input intensity and s = output intensity



Example:
Contrast
enhancement

Grey level transformation (also known as intensity transformation) is a fundamental spatial domain technique in digital image processing where each pixel's intensity value is directly modified to produce an enhanced output image.

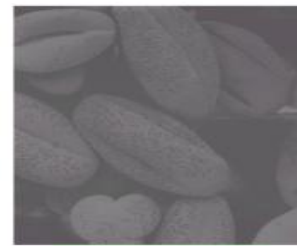
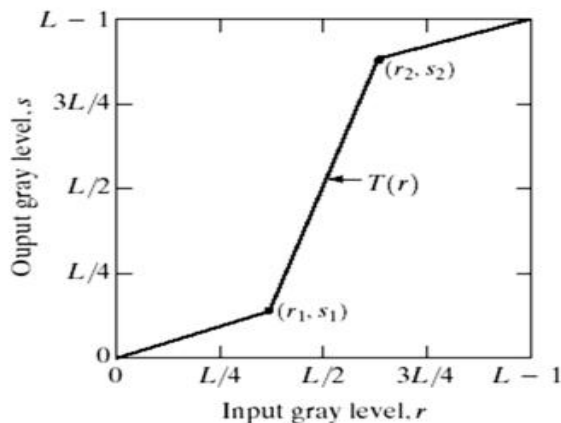
Contrast Stretching:

The contrast stretching transformation is used to improve details of images that were acquired using poor illumination conditions, an image sensor with a less dynamic range, or the wrong camera lens setting. The output of this transformation is an image with higher contrast than the original image, which

is achieved by darkening levels below gray level r_1 and brightening levels above r_2 in the original image. Mathematically, this transformation is given as follows:

Contrast Stretching

Contrast means the difference between the brightest and darkest intensities



Before contrast enhancement



After

Basic Grey Level Transformations

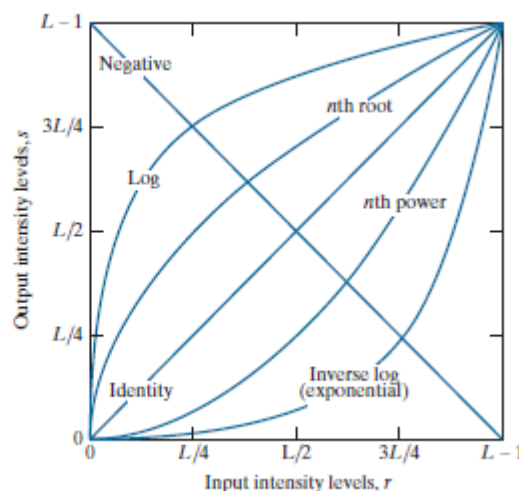
IMAGE NEGATIVES

The negative of an image with intensity levels in the range $[0, L-1]$ is obtained by using the negative transformation function shown in Fig. 3.3, which has the form:

$$s = L - 1 - r$$

Reversing the intensity levels of a digital image in this manner produces the equivalent of a photographic negative. This type of processing is used, for example, in enhancing white or gray detail embedded in dark regions of an image, especially when the black areas are dominant in size.

FIGURE 3.3
Some basic intensity transformation functions. Each curve was scaled independently so that all curves would fit in the same graph. Our interest here is on the shapes of the curves, not on their relative values.



Where $0 \leq r \leq L - 1$, $0 \leq S \leq L - 1$, L being the total number of gray levels. If the image is an 8-bit image, then the gray level L value = 256, and therefore $0 \leq r \leq 255$, $0 \leq S \leq 255$.

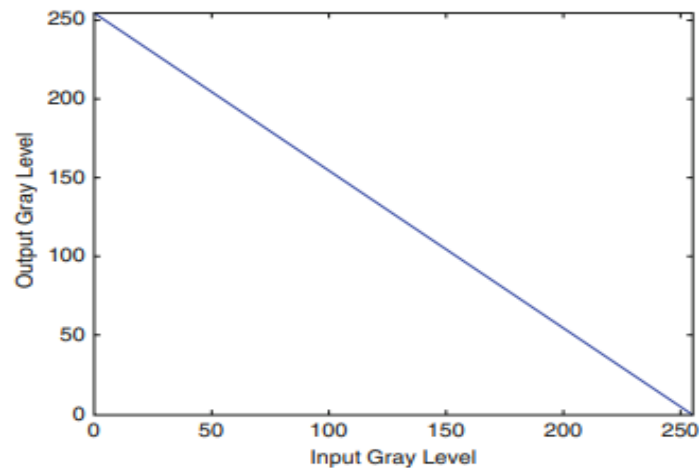
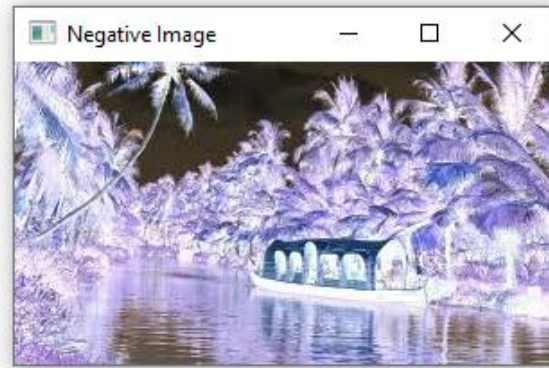


Fig. 2.3 Image negative transformation

Example: Python code for image negative:

Install the library using `pip install opencv-python`.

```
import cv2
import numpy as np
# Load the image (use 0 for grayscale, or omit for color image)
img = cv2.imread('d:/download.jpg')
# Create the negative image
# Subtract each pixel value from the maximum possible value (255 for 8-bit images)
img_neg = 255 - img
# Display the original and negative images
cv2.imshow('Original Image', img)
cv2.imshow('Negative Image', img_neg)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Log Transformation: During daytime, we can't observe the stars in the sky due to the presence of the sun and its high intensity. In the same way, it is suggested that our eye cannot see images with high pixel values. Thus, some pixel values must lie in a certain meaningful range in order to be obscured.

Log transformation is used to compress the high pixel values so that the user can see the image clearly. The log operator is an excellent compressing function and is therefore used in this transformation. It expands the values of dark pixels and compresses the high pixel values.

Mathematically, this transformation is given as follows:

$$S = c * \log(1+r)$$

where c is a constant and it is assumed that $r \geq 0$. The shape of the log curve in Fig. 3.3(above graph) shows that this transformation maps a narrow range of low intensity values in the input into a wider range of output levels. Graphically, this transformation can be viewed in Fig.

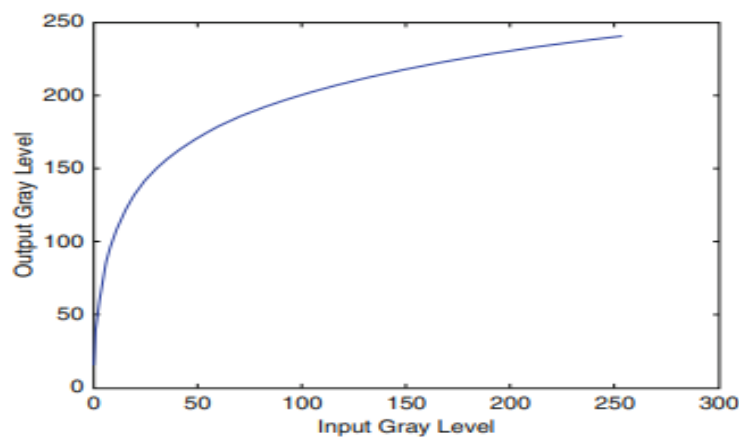


Fig. 2.12 Log transformation

Example: log transformations

```
import cv2
import numpy as np

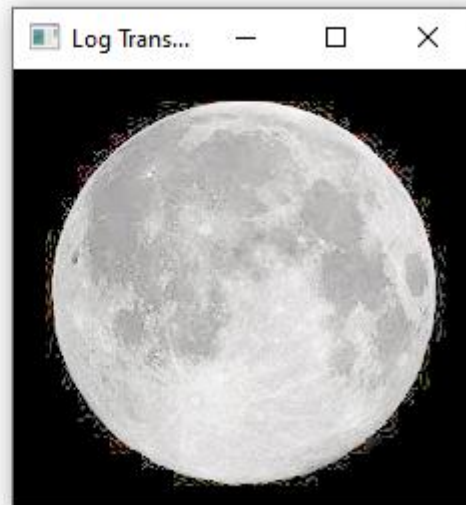
# Load the image
image = cv2.imread('d:/moon.jpg')

# 1. Calculate the scaling constant 'c'
#  $c = 255 / \log(1 + \max\_pixel\_value)$  to ensure result fits in 8-bit [0-255]
c = 255 / np.log(1 + np.max(image))

# 2. Apply the log transformation formula
# We add 1 to avoid  $\log(0)$ , which is undefined
log_image = c * (np.log(image + 1))

# 3. Convert the float values back to 8-bit unsigned integers
log_image = np.array(log_image, dtype=np.uint8)

# Display results
cv2.imshow('Original Image', image)
cv2.imshow('Log Transformed', log_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Power Low Transformation

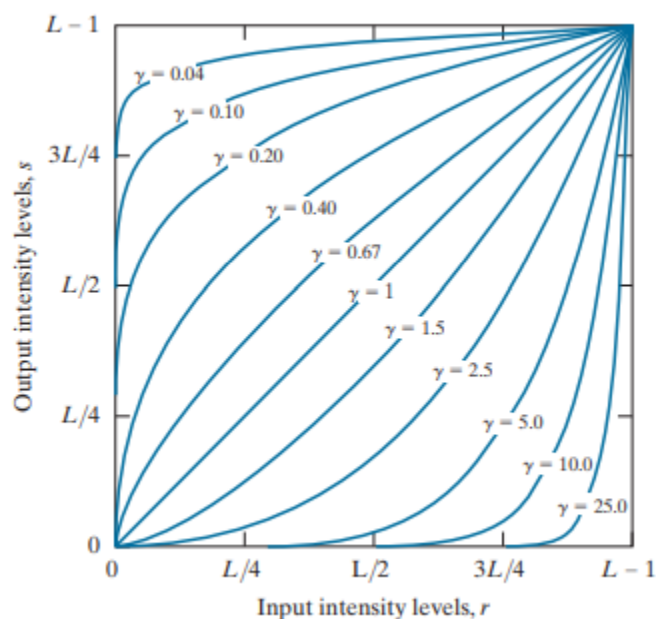
The power law transformation maps a narrow range of dark input values into a wider range of output values or vice versa. Varying γ gives a whole family of curves. Mathematically, this transformation is given as follows:

Power-law transformations have the form

$$s = cr^\gamma$$

where c and γ are positive constants. Graphically, this transformation can be viewed in Fig. This method is widely used in television for the correction of video representations using cathode ray tubes (CRTs).

FIGURE 3.6
Plots of the gamma equation $s = cr^\gamma$ for various values of γ ($c = 1$ in all cases). Each curve was scaled independently so that all curves would fit in the same graph. Our interest here is on the *shapes* of the curves, not on their relative values.



- ▶ If gamma < 1: the mapping is weighted toward brighter output values.
- ▶ If gamma = 1 :the mapping is linear
- ▶ If gamma > 1 :the mapping is weighted toward darker output values.

Example:

```
import cv2
```

```
import numpy as np
```

```
# Load the image
```

```
# Make sure to have an image named 'input_image.jpg' in the same directory
```

```
img = cv2.imread('d:/butterfly.jpg')
```

Convert to float and normalize to the range [0, 1]

```
img_normalized = img / 255.0
```

Apply the power law transformation

```
gamma = 0.6 # Adjust gamma value: < 1 brightens, > 1 darkens
```

```
img_transformed_normalized = cv2.pow(img_normalized, gamma)
```

Rescale back to the range [0, 255] and convert to uint8

```
img_transformed = np.array(255 * img_transformed_normalized, dtype='uint8')
```

Display the original and transformed images

```
cv2.imshow('Original Image', img)
```

```
cv2.imshow(f'Power Law Transformation (Gamma={gamma})', img_transformed)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```



Power-Law Transformations: Gamma Correction Application



Original image
MR of fractured
human spine

$$S = I^{0.6}$$

MRI Image after
Gamma Correction

$$S = cr^\gamma$$

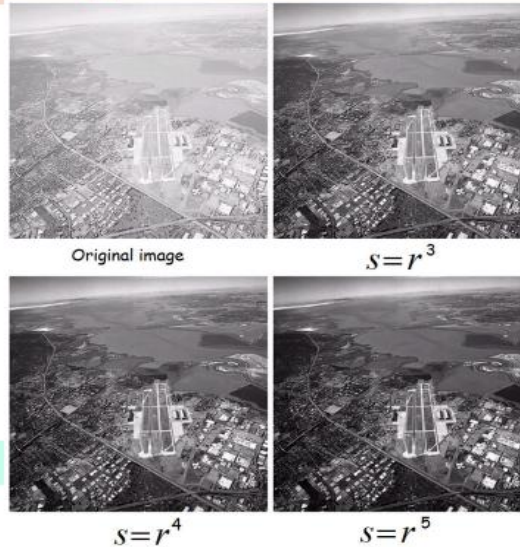


$$S = I^{0.4}$$

$$S = I^{0.3}$$

Power-Law Transformations: Gamma Correction Application

$$S = cr^\gamma$$



Ariel images after
Gamma Correction

```
J = imadjust(I,[],[],gamma);
```

Histogram Processing:

Histogram processing is a fundamental technique in digital image processing used for **analyzing and enhancing image contrast**. It involves manipulating the histogram of pixel intensity values to improve visual quality or extract useful information. Histogram manipulation is a fundamental tool in image processing. Histograms are simple to compute and are also suitable for fast hardware implementations, thus making histogram-based techniques a popular tool for real-time image processing. Histogram shape is related to image appearance.

Example:

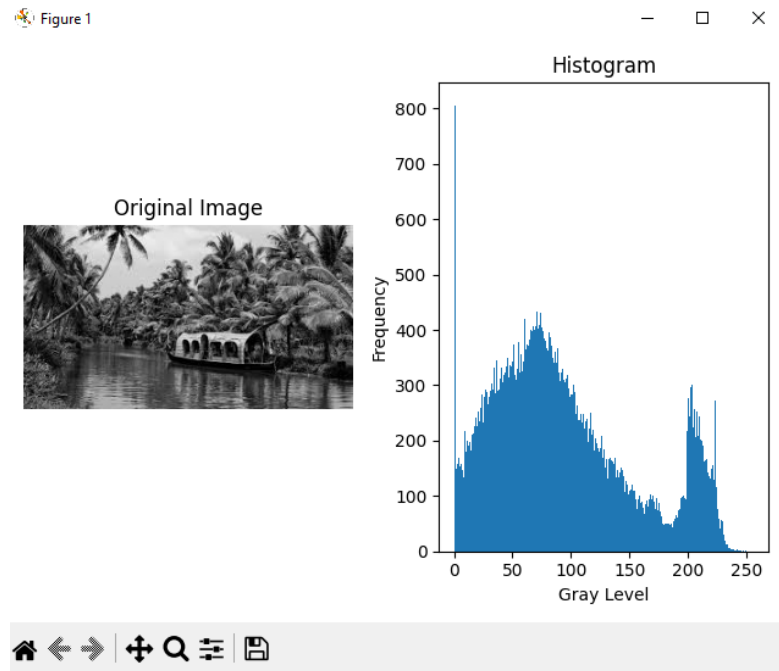
```
import cv2
import matplotlib.pyplot as plt

# Load grayscale image
img = cv2.imread('d:/image.jpg', 0)

# Display image
plt.subplot(1,2,1), plt.imshow(img, cmap='gray')
plt.title('Original Image'), plt.axis('off')

# Plot histogram
plt.subplot(1,2,2), plt.hist(img.ravel(), bins=256, range=[0,256])
```

```
plt.title('Histogram'), plt.xlabel('Gray Level'), plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```



Histogram processing can be used for image enhancement, image compression, and image segmentation and can be divided into two types:

- **Histogram stretching:** Histogram stretching (also called **contrast stretching** or **normalization**) is a technique used to improve the contrast of an image by expanding the range of intensity values. Instead of compressing or redistributing values like histogram equalization, stretching linearly maps pixel values to a wider range.
- **Histogram equalization:** Histogram equalization is a digital image processing technique used to improve image contrast by redistributing pixel intensity values across the available range. It works by transforming the image's histogram so that intensities are spread more evenly, making details in darker or lighter regions more visible.

Histogram Equalization process fundamental of spatial filtering

In histogram equalization, the main idea is to equalize all values of the histogram so that the entire gray level range from 0 to $L - 1$ is covered. Transformation for this process may be as follows:

$$S = T(r), 0 \leq r \leq 1 \quad (2.16)$$

The following conditions can be used to satisfy Eq. 2.16:

- $T(r)$ is a single-valued and monotonically increasing function in the interval $0 \leq r \leq 1$.
- $0 \leq T(r) \leq 1$ for $0 \leq r \leq 1$.

The first condition is needed to guarantee that the inverse transformation will exist and monotonicity ensures that no gray level is inverted. The second condition ensures that the output gray levels will be in the same range as the input levels. The inverse transformation from S back to r is given as follows:

$$r = T^{-1}(S), 0 \leq S \leq 1 \quad (2.17)$$

Histogram Equalization

For Histogram of Digital Image, we use

$$\begin{aligned} s_k = T(r_k) &= \sum_{j=0}^k p_r(r_j) \\ &= \sum_{j=0}^k \frac{n_j}{N} \end{aligned}$$

n_j = the number of pixels with intensity = j
 N = the number of total pixels

The gray levels in the image can be viewed as random variables in the interval [0, 1]. They can be characterized by their probability density function (PDF).

$$S_k = T(r_k) = \sum_{i=0}^k P_r(r_j) = \sum_{i=0}^k \frac{n_i}{n}, \text{ for } k = 0, 1, \dots, L-1 \quad (2.22)$$

Equation 2.22 is called histogram equalization.

Example of Histogram Equalization: Table 2.3 shows the numbers of pixels for an image with gray levels in the range [0, 7]. Solution for Histogram Equalization A solution for histogram equalization is shown in Tables 2.4 and 2.5

Table 2.3 Gray levels in the range [0, 7]

Gray level	0	1	2	3	4	5	6	7
No. of pixels	790	1023	850	656	329	245	122	81

Table 2.4 Histogram equalization process on given gray level values

Gray level	n_k	PDF	CDF	$(L-1) \times S_k$	Rounding off
0	790	0.19	0.19	1.33	1
1	1023	0.25	0.44	3.08	3
2	850	0.21	0.65	4.55	5
3	656	0.16	0.81	5.67	6
4	329	0.08	0.89	6.23	6
5	245	0.06	0.95	6.65	7
6	122	0.03	0.98	6.86	7
7	81	0.02	1	7	7
Total	4096				

PDF, Probability Density Function

CDF, Cummulative Density Function

Table 2.5 Output of histogram equalization process

Equalized gray level	0	1	2	3	4	5	6	7
No. of pixels	0	790	0	1023	0	850	985	448

Python code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image in grayscale
img = cv2.imread('d:/beachflower.jpg', cv2.IMREAD_GRAYSCALE)
```

Apply Histogram Equalization

```
equalized_img = cv2.equalizeHist(img)
```

Plot original and equalized images

```
plt.figure(figsize=(12,8))
```

```
plt.subplot(2,2,1)
```

```
plt.title("Original Image")
```

```
plt.imshow(img, cmap='gray')
```

```
plt.axis('off')
```

```
plt.subplot(2,2,2)
```

```
plt.title("Equalized Image")
```

```
plt.imshow(equalized_img, cmap='gray')
```

```
plt.axis('off')
```

Plot histograms

```
plt.subplot(2,2,3)
```

```
plt.title("Original Histogram")
```

```
plt.hist(img.ravel(), bins=256, range=[0,256], color='blue')
```

```
plt.xlabel("Pixel Intensity")
```

```
plt.ylabel("Frequency")
```

```
plt.subplot(2,2,4)
```

```
plt.title("Equalized Histogram")
```

```
plt.hist(equalized_img.ravel(), bins=256, range=[0,256], color='green')
```

```
plt.xlabel("Pixel Intensity")
```

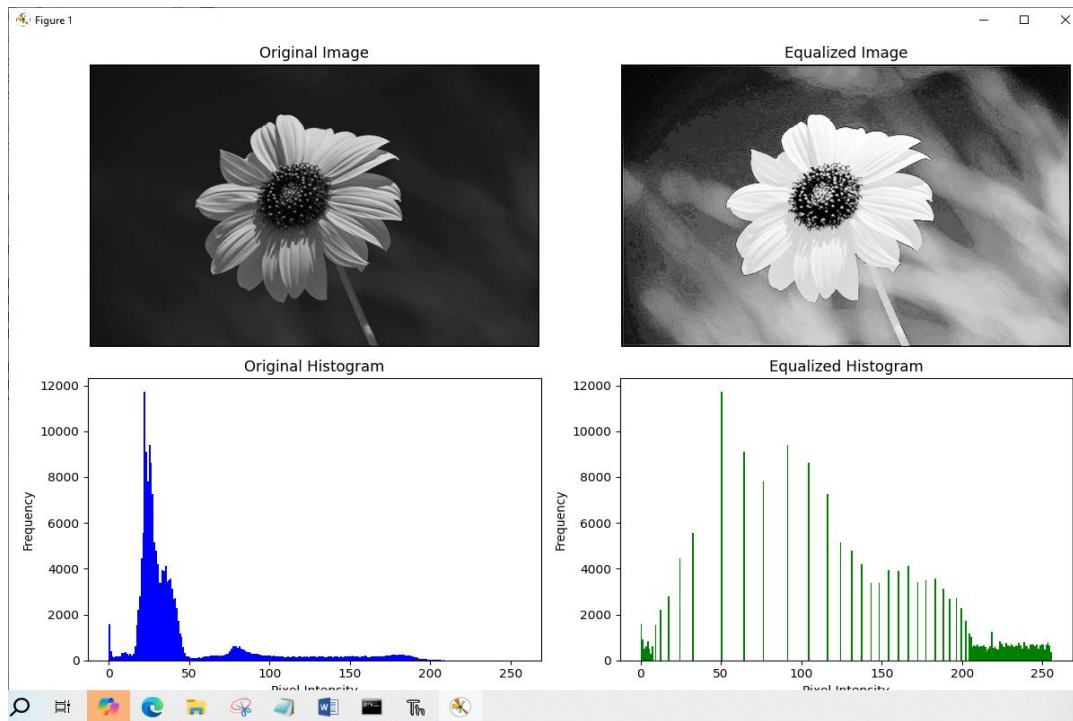
```
plt.ylabel("Frequency")
```

```
plt.tight_layout()
```

```
plt.show()
```

Save the equalized image

```
cv2.imwrite('equalized_image.jpg', equalized_img)
```



Correlation and convolution spatial filtering:

Correlation and convolution are foundational spatial filtering techniques in image processing, used to apply filters (kernels) to images for tasks like blurring, sharpening, and edge detection. The key difference lies in how the kernel is applied—convolution flips the kernel before sliding, while correlation does not.

Correlation:

The operation called *correlation* is closely related to convolution. In correlation, the value of an output pixel is also computed as a weighted sum of neighboring pixels. The difference is that the matrix of weights, in this case called the *correlation kernel*, is not rotated during the computation. The Image Processing Toolbox™ filter design functions return correlation kernels.

For example, suppose the image is

$$A = \begin{pmatrix} 17 & 24 & 1 & 8 & 15 \\ 23 & 5 & 7 & 14 & 16 \\ 4 & 6 & 13 & 20 & 22 \\ 10 & 12 & 19 & 21 & 3 \\ 11 & 18 & 25 & 2 & 9 \end{pmatrix}$$

and the correlation kernel is

$$h = \begin{bmatrix} 8 & 1 & 6 \\ 3 & 5 & 7 \\ 4 & 9 & 2 \end{bmatrix}$$

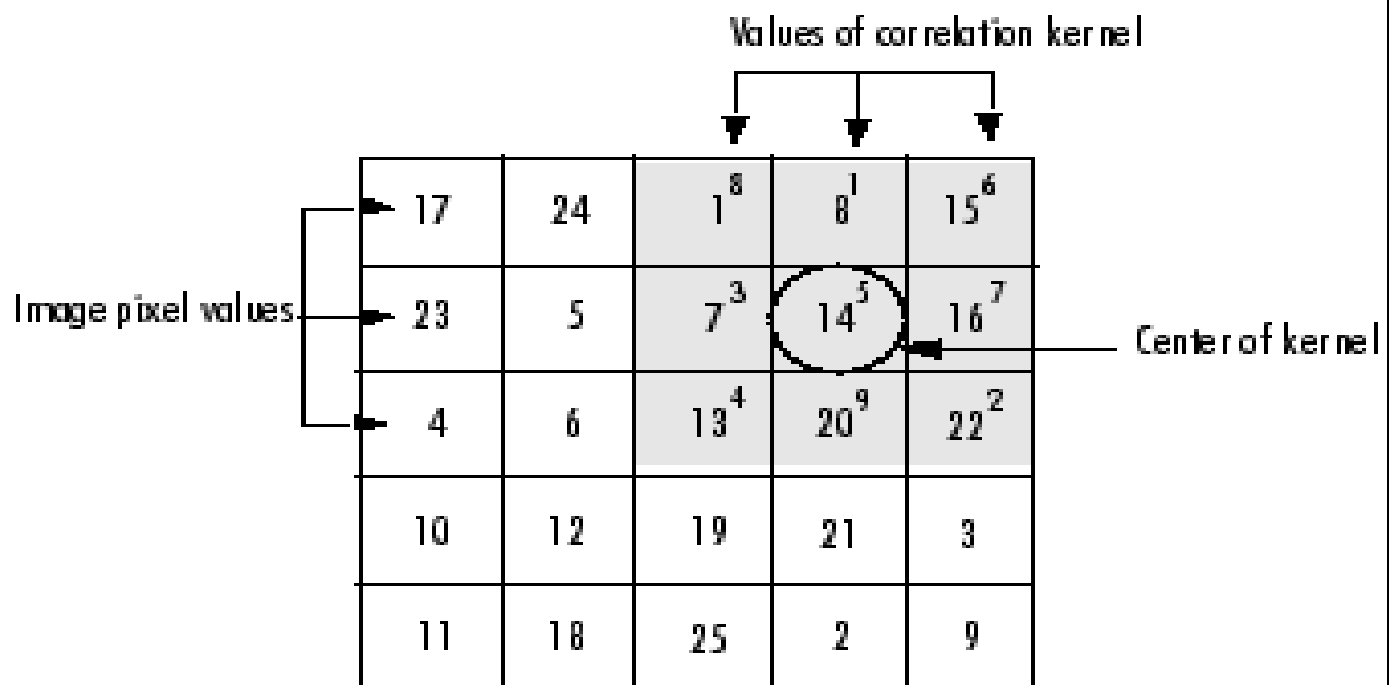
The following figure shows how to compute the (2,4)(2nd row 4th column pixel) output pixel of the correlation of A, assuming h is a correlation kernel instead of a convolution kernel, using these steps:

- Slide the center element of the correlation kernel so that lies on top of the (2,4)(2nd row 4th column pixel) element of A.
- Multiply each weight in the correlation kernel by the pixel of A underneath.
- Sum the individual products.

The (2,4) output pixel from the correlation is

$$1 \cdot 8 + 8 \cdot 1 + 15 \cdot 6 + 7 \cdot 3 + 14 \cdot 5 + 16 \cdot 7 + 13 \cdot 4 + 20 \cdot 9 + 22 \cdot 2 = 585$$

Computing the (2,4) Output of Correlation



Convolution

Linear filtering of an image is accomplished through an operation called *convolution*. Convolution is a neighborhood operation in which each output pixel is the weighted sum of neighboring input pixels. The matrix of weights is called the *convolution kernel*, also known as the *filter*. A convolution kernel is a correlation kernel that has been rotated 180 degrees.

For example, suppose the image is

```
A = [17      24      1      8      15
      23      5      7     14     16
      4       6      13    20    22
      10      12      19    21     3
      11      18    25         2       9]
```

and the correlation kernel is

```
h = [8   1   6
      3   5   7
      4   9   2]
```

You would use the following steps to compute the output pixel at position (2,4)(2nd row 4th column pixel):

1. Rotate the correlation kernel 180 degrees about its center element to create a convolution kernel.

```
h=  [2      9      4
      7      5      3
      6      1      8]
```

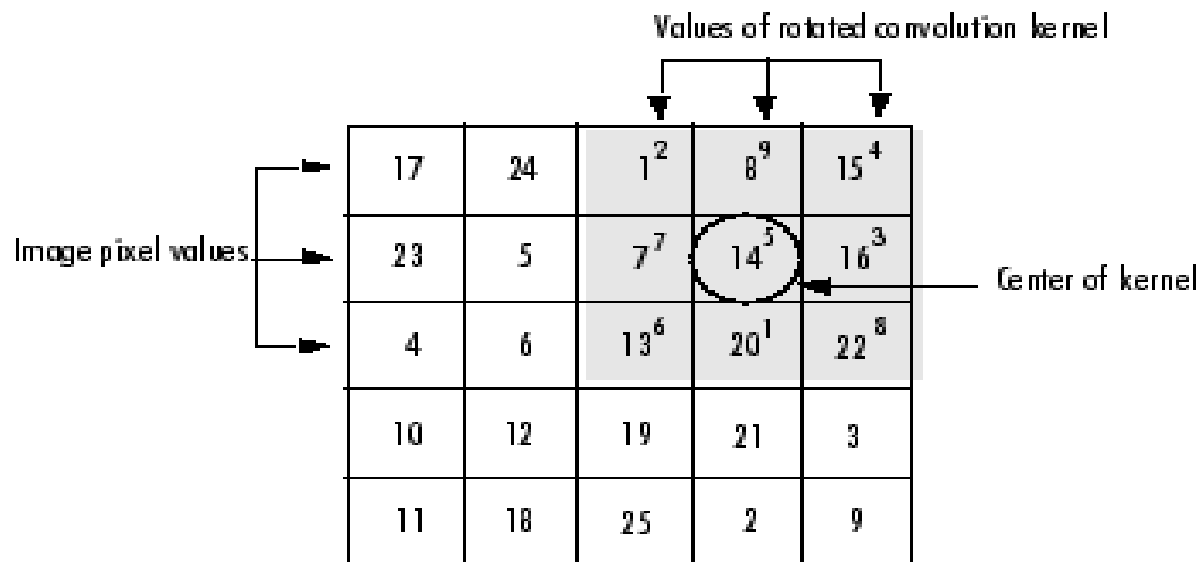
2. Slide the center element of the convolution kernel so that it lies on top of the (2,4)(2nd row 4th column pixel) element of A.
3. Multiply each weight in the rotated convolution kernel by the pixel of A underneath.
4. Sum the individual products from step 3.

Hence the (2,4) output pixel is

$$1 \cdot 2 + 8 \cdot 9 + 15 \cdot 4 + 7 \cdot 7 + 14 \cdot 5 + 16 \cdot 3 + 13 \cdot 6 + 20 \cdot 1 + 22 \cdot 8 = 575$$

Shown in the following figure.

Computing the (2,4) Output of Convolution



Mask for low pass-filtering(smoothing/blurring)

The Python code given below applies low-pass average filters to the test image to create a blurring effect. The script uses linear convolution with a kernel filled with ones divided by the kernel size.

```
import cv2
import numpy as np
import urllib.request
import os

def main():
    # Download a sample image
    url = "https://upload.wikimedia.org/wikipedia/en/7/7d/Lenna_%28test_image%29.png"
    input_path = "input_image.png"
    if not os.path.exists(input_path):
        print("Downloading sample image...")
        req = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
        with urllib.request.urlopen(req) as response, open(input_path, 'wb') as out_file:
            out_file.write(response.read())

    # Read the image
```

```

image = cv2.imread(input_path)
if image is None:
    print("Error: Could not read image.")
    return

# 1. Low-Pass Average Filter Kernel (3x3)
# All elements are 1/9. This averages the pixels in a 3x3 neighborhood.
kernel_size = 3
low_pass_kernel_3x3 = np.ones((kernel_size, kernel_size), np.float32) /
(kernel_size * kernel_size)

# 2. Low-Pass Average Filter Kernel (5x5, for stronger blur/smoothing)
# All elements are 1/25.
kernel_size_5 = 5
low_pass_kernel_5x5 = np.ones((kernel_size_5, kernel_size_5), np.float32) /
(kernel_size_5 * kernel_size_5)

# Apply the filters using cv2.filter2D
low_pass_3x3_img = cv2.filter2D(image, -1, low_pass_kernel_3x3)
low_pass_5x5_img = cv2.filter2D(image, -1, low_pass_kernel_5x5)

# Save outputs
cv2.imwrite("output_low_pass_3x3.png", low_pass_3x3_img)
cv2.imwrite("output_low_pass_5x5.png", low_pass_5x5_img)

print("Process Complete!")
print(f"Input image saved as: input_image.png")
print(f"Low-pass filter (3x3 average) saved as: output_low_pass_3x3.png")
print(f"Low-pass filter (5x5 average) saved as: output_low_pass_5x5.png")

if __name__ == "__main__":
    main()

```

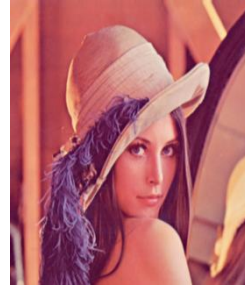
Result:



Input image



Low-pass filter(3x3)



Low-pass filter(5x5)

Low-Pass Median Filter (Order Statistic Filter)

We can see from Fig. 2.24 that the low-pass average filter removes noise by blurring it until it is no longer seen. But this process also blurs the edges. Hence, by using an averaging mask, more blurring can be available in the filtered image. So, rather than an averaging process, a non-linear filter known as a median filter is used. This is also known as an order statistics filter because its response is based on the ordering or ranking of the pixels contained within the mask. The steps for this filter are as follows:

- Assume a 3×3 empty matrix.
- Place the mask at the left-hand corner.
- Arrange nine pixels in ascending or descending order.
- Choose the middle (median) value of the nine values.
- Place the value at the center.
- Move the mask in a similar fashion to the averaging filter until the end of the image.

```
import cv2
import numpy as np
import urllib.request
import os
def main():
    # 1. First, make sure we have a sample image to work with.
    input_path = r"E:\Professional Files\4th sem BCA\rosecolor.jpg"
```

```

if not os.path.exists(input_path):
    print(f'Error: Could not find image at {input_path}')
    return
# Read the image
image = cv2.imread(input_path)
if image is None:
    print("Error: Could not read image.")
    return

# Apply Low-Pass Median Filter (Order Statistic Filter)
# The cv2.medianBlur() function takes the image and kernel size.
# The kernel size must be a positive, odd integer (e.g., 3, 5, 7, etc.)
# The function evaluates a sliding neighborhood, sorts the pixel values,
# and replaces the central pixel with the median value.
# 1. Median Filter (3x3 Kernel)
kernel_size_3 = 3
median_pass_3x3_img = cv2.medianBlur(image, kernel_size_3)
# 2. Median Filter (5x5 Kernel, for stronger smoothing/noise reduction)
kernel_size_5 = 5
median_pass_5x5_img = cv2.medianBlur(image, kernel_size_5)
# Save outputs
cv2.imwrite("output_median_pass_3x3.png", median_pass_3x3_img)
cv2.imwrite("output_median_pass_5x5.png", median_pass_5x5_img)
print("Process Complete!")
print(f'Input image saved as: input_image.png')
print(f'Median filter (3x3 kernel) saved as: output_median_pass_3x3.png')
print(f'Median filter (5x5 kernel) saved as: output_median_pass_5x5.png')
if __name__ == "__main__":
    main()

```

High-Pass Filter (sharpening)

The main objective of sharpening is to highlight fine details or enhance details that have been blurred. The sum of the elements of the high-pass mask must equal zero. This is because when we place the mask on the low-frequency region (i.e., over an area of constant gray level), the result must be zero. The filter reduces the average gray level of the image to zero. This means that there are some negative gray levels in the output of the filter. Thus, some form of scaling is required.

Python Code for High Pass Filter

```
import cv2
import numpy as np
import urllib.request
import os

def main():
    # Download a sample image
    url = "https://upload.wikimedia.org/wikipedia/en/7/7d/Lenna_%28test_image%29.png"
    input_path = "input_image.png"
    if not os.path.exists(input_path):
        print("Downloading sample image...")
        req = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
        with urllib.request.urlopen(req) as response, open(input_path, 'wb') as out_file:
            out_file.write(response.read())

    # Read the image
    image = cv2.imread(input_path)
    if image is None:
        print("Error: Could not read image.")
```

```

    return

# 1. High-Pass Filter Kernel (Extracts Edges)
# The sum of all elements is 0 (prevents overall brightness increase)
high_pass_kernel = np.array([[ -1, -1, -1],
                             [ -1,  8, -1],
                             [ -1, -1, -1]])

# 2. Sharpening Kernel (Original Image + High-Pass Filter)
# The sum is 1 (keeps the original image brightness intact)
sharpen_kernel = np.array([[ -1, -1, -1],
                           [ -1,  9, -1],
                           [ -1, -1, -1]])

# Apply the filters using cv2.filter2D for 2D convolution
# -1 means the output image will have the same depth as the source
high_pass_img = cv2.filter2D(image, -1, high_pass_kernel)
sharpened_img = cv2.filter2D(image, -1, sharpen_kernel)

# Save outputs
cv2.imwrite("output_high_pass.png", high_pass_img)
cv2.imwrite("output_sharpened.png", sharpened_img)

print("Process Complete!")
print(f"Input image saved as: input_image.png")
print(f"High-pass filter only (edges) saved as: output_high_pass.png")
print(f"Sharpened image saved as: output_sharpened.png")

if __name__ == "__main__":
    main()

```

This Python script applies a high-pass filter to extract edges and a sharpening filter to enhance an image.

Script Details

The script uses opencv-python to apply linear convolution (`cv2.filter2D`) with two kernels:

1. **High-Pass (Edge Extraction):**

```
[[ -1, -1, -1],  
 [ -1,  8, -1],  
 [ -1, -1, -1]]
```

2. **Sharpening (Original + High-Pass):**

```
[[ -1, -1, -1],  
 [ -1,  9, -1],  
 [ -1, -1, -1]]
```

Results



Input Image



High-Pass Filter Output (Edges)



Sharpened Image Output

Filter Comparison Grid

To make it easier to see and compare the effects of each filter, I wrote a visualization script using Matplotlib that plots the original image alongside all the processed versions in a single combined grid.

Visual Comparison Matrix



Enhancement Using Arithmetic/Logic Operations

1. **Image Subtraction:** The difference between two images $f(x, y)$ and $h(x, y)$, expressed as

$$g(x, y) = f(x, y) - h(x, y),$$

is obtained by computing the difference between all pairs of corresponding pixels from f and h . The key usefulness of subtraction is the enhancement of differences between images. Figure 3.28(a) shows the fractal image used earlier to illustrate the concept of bit planes. Figure 3.28(b) shows the result of discarding (setting to zero) the four least significant bit planes of the original image.

The images are nearly identical visually, with the exception of a very slight drop in overall contrast due to less variability of the gray level values in

the image of Fig.3.28(b). The pixel-by-pixel difference between these two images is shown in Fig.3.28(c).

The differences in pixel values are so small that the difference image appears nearly black when displayed on an 8-bit Display. We chose histogram equalization, but an appropriate power-law transformation would have done the job also. The result is shown in Fig.3.28(d). This is a very useful image for

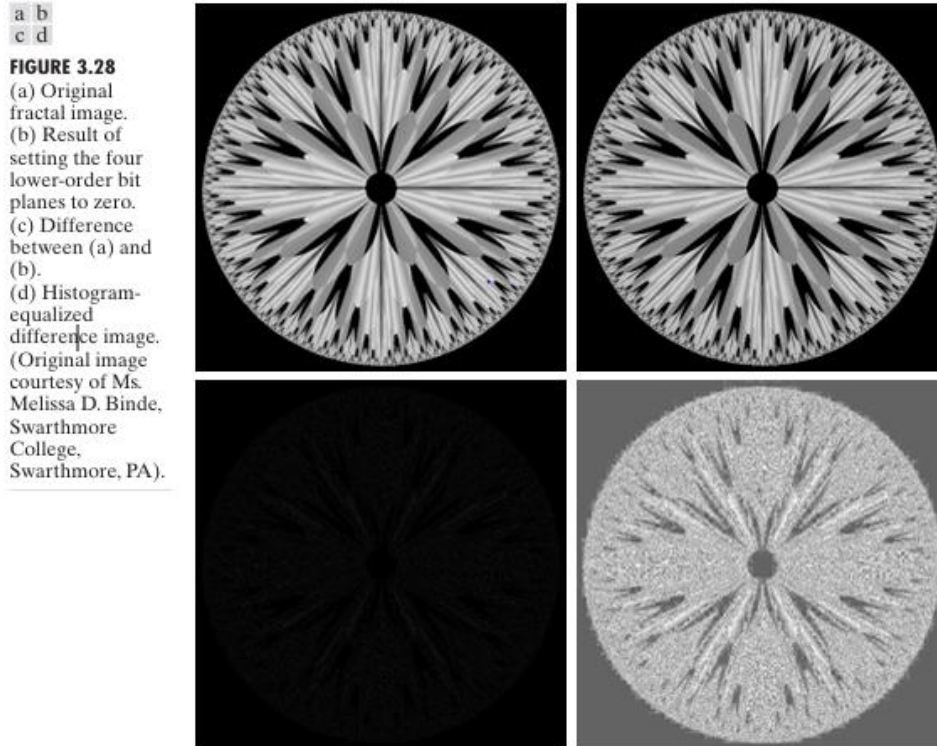


FIGURE 3.28
(a) Original fractal image.
(b) Result of setting the four lower-order bit planes to zero.
(c) Difference between (a) and (b).
(d) Histogram-equalized difference image.
(Original image courtesy of Ms. Melissa D. Binde, Swarthmore College, Swarthmore, PA).

evaluating the effect of setting to zero the lower-order planes.

2. Image Averaging : Consider a noisy image $g(x, y)$ formed by the addition of noise $h(x, y)$ to an original image $f(x, y)$; that is,

$$g(x, y) = f(x, y) + h(x, y)$$

where the assumption is that at every pair of coordinates (x, y) the noise is uncorrelated[†] and has zero average value. The objective of the following procedure is to reduce the noise content by adding a set of noisy images, $\{g_i(x, y)\}$.

If the noise satisfies the constraints just stated, if an image $g(x, y)$ is formed by averaging K different noisy images,

$$\bar{g}(x, y) = \frac{1}{K} \sum_{i=1}^K g_i(x, y)$$

then it follows that

$$E\{\bar{g}(x, y)\} = f(x, y) \quad (3.4-4)$$

and

$$\sigma_{\bar{g}(x, y)}^2 = \frac{1}{K} \sigma_{\eta(x, y)}^2 \quad (3.4-5)$$

where $E\{\bar{g}(x, y)\}$ is the expected value of $\bar{g}$, and $\sigma_{\bar{g}(x, y)}^2$ and $\sigma_{\eta(x, y)}^2$ are the variances of $\bar{g}$ and η , all at coordinates (x, y) . The standard deviation at any point in the average image is

$$\sigma_{\bar{g}(x, y)} = \frac{1}{\sqrt{K}} \sigma_{\eta(x, y)}. \quad (3.4-6)$$

As K increases, Eqs. (3.4-5) and (3.4-6) indicate that the variability (noise) of the pixel values at each location (x, y) decreases. Because $E\{\bar{g}(x, y)\} = f(x, y)$, this means that $\bar{g}(x, y)$ approaches $f(x, y)$ as the number of noisy images used in the averaging process increases. In practice, the images $g_i(x, y)$ must be registered (aligned) in order to avoid the introduction of blurring and other artifacts in the output image.

FUNDAMENTALS OF SPATIAL FILTERING:

Spatial filtering is used in a broad spectrum of image processing applications, so a solid understanding of filtering principles is important. The name filter is borrowed from frequency domain where “filtering” refers to passing, modifying, or rejecting specified frequency components of an image.

For example, a filter that passes low frequencies is called a low pass filter. The net effect produced by a low pass filter is to smooth an image by blurring it. We can accomplish similar smoothing directly on the image itself by using spatial filters.

Spatial filtering modifies an image by replacing the value of each pixel by a function of the values of the pixel and its neighbors. If the operation performed on the image pixels is linear, then the filter is called a linear spatial filter. Otherwise, the filter is a nonlinear spatial filter.

THE MECHANICS OF LINEAR SPATIAL FILTERING

A linear spatial filter performs a sum-of-products operation between an image f and a filter kernel, w . The kernel is an array whose size defines the

neighborhood of operation, and whose coefficients determine the nature of the filter. Other terms used to refer to a spatial filter kernel are mask, template, and window. We use the term filter kernel or simply kernel.

In general, linear spatial filtering of an image of size $M \times N$ with a kernel of size $m \times n$ is given by the expression

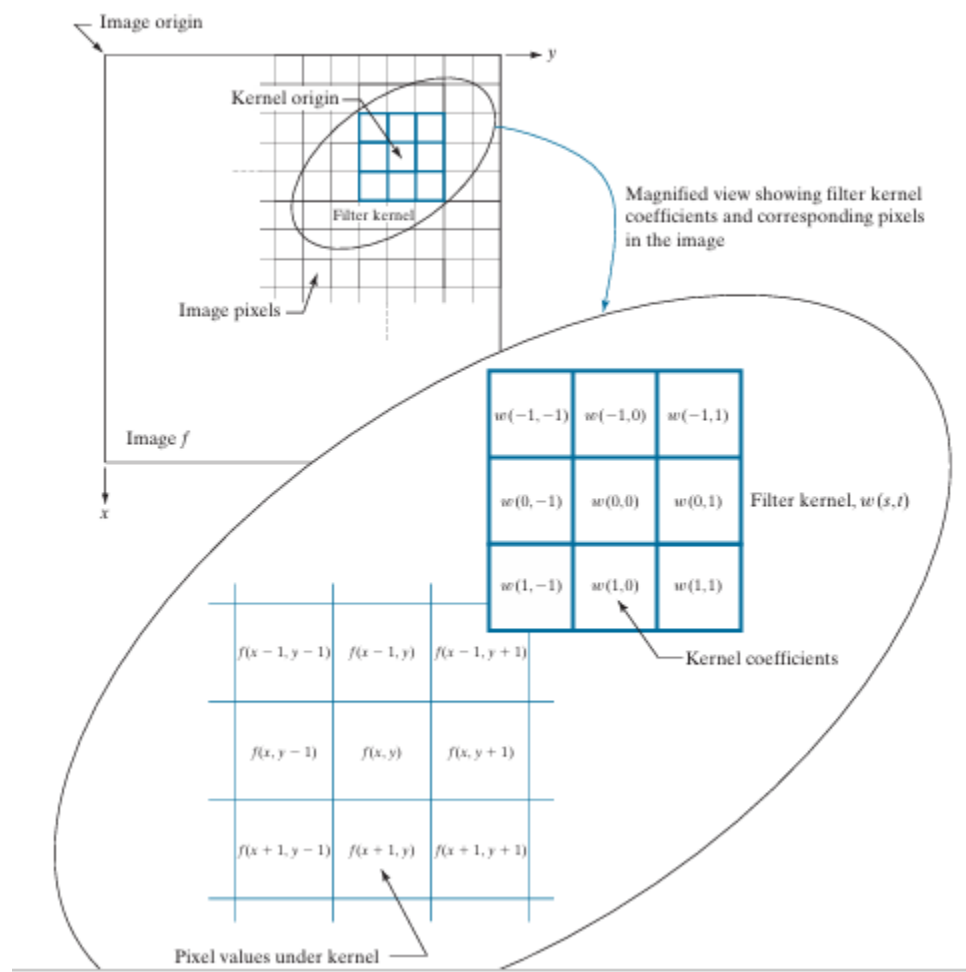
$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

where x and y are varied so that the center (origin) of the kernel visits every pixel in f once.

SPATIAL CORRELATION AND CONVOLUTION:

Spatial correlation is illustrated graphically in Fig. 3.28, and it is described mathematically by

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$



Correlation consists of moving the center of a kernel over an image, and computing the sum of products at each location. The mechanics of spatial convolution are the same, except that the correlation kernel is rotated by 180°. Thus, when the values of a kernel are symmetric about its center, correlation and convolution yield the same result.

SMOOTHING (LOWPASS) SPATIAL FILTERS: Smoothing (also called averaging) spatial filters are used to reduce sharp transitions in intensity. Because random noise typically consists of sharp transitions in intensity, an obvious application of smoothing is noise reduction. Smoothing is used to reduce irrelevant detail in an image, where “irrelevant” refers to pixel regions that are small with respect to the size of the filter kernel. Another application is for smoothing the false contours that result from using an insufficient number of intensity levels in an image. Smoothing filters are used in combination with other techniques for image enhancement, such as the histogram processing techniques and unsharp masking,

Linear spatial filtering consists of convolving an image with a filter kernel. Convolving a smoothing kernel with an image blurs the image, with the degree of blurring being determined by the size of the kernel and the values of its coefficients. In addition to being useful in countless applications of image processing, low pass filters are fundamental, in the sense that other important filters, including sharpening (high pass), bandpass, and band reject filters, can be derived from low pass filters.

EXAMPLE:

```
import cv2
import numpy as np

# Read the image
img = cv2.imread('d:/nature.jpg', 0) # Read in grayscale

# Define a 3x3 averaging kernel
kernel = np.ones((3, 3), np.float32) / 9

# Apply the filter using filter2D
blurred = cv2.filter2D(img, -1, kernel)
cv2.imshow('Blurred', blurred)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Original image:



Blurred Image:



‘ORDER-STATISTIC (NONLINEAR) FILTERS : Order-statistic filters are nonlinear spatial filters whose response is based on ordering (ranking) the pixels contained in the region encompassed by the filter. Smoothing is achieved by replacing the value of the center pixel with the value determined by the ranking result. The best-known filter in this category is the median filter, which, as its name implies, replaces the value of the center pixel by the median of the intensity values in the neighborhood of that pixel (the value of the center pixel is included).

Median filters provide excellent noise reduction capabilities for certain types of random noise, with considerably less blurring than linear smoothing filters of similar size. Median filters are particularly effective in the presence of impulse noise (sometimes called salt-and-pepper noise, when it manifests itself as white and black dots superimposed on an image).

Example:

```
import cv2
img = cv2.imread('d:/noiseimage.jpg', 0)
# Apply Median Blur with a 5x5 aperture
median_blur = cv2.medianBlur(img, 5)
cv2.imshow('Median Blur', median_blur)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Original image:

Median Blur Image:



SHARPENING (HIGHPASS) SPATIAL FILTERS: Sharpening highlights transitions in intensity. Uses of image sharpening range from electronic printing and medical imaging to industrial inspection and autonomous guidance in military systems.

Averaging is analogous to integration, it is logical to conclude that sharpening can be accomplished by spatial differentiation. . The strength of the response of a derivative operator is proportional to the magnitude of the intensity discontinuity at the point at which the operator is applied. Thus, image differentiation enhances edges and other discontinuities (such as noise) and de-emphasizes areas with slowly varying intensities

Example;

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

1. Load the image in grayscale

```
image = cv2.imread('d:/image.jpg', cv2.IMREAD_GRAYSCALE)
```

2. Define Sharpening Kernel (High-Pass Filter)

```
# Center element is high, neighbors are negative
```

```
kernel = np.array([[ 0, -1,  0],
                   [-1,  5, -1],
                   [ 0, -1,  0]])
```

3. Apply the filter using filter2D

```
sharpened_image = cv2.filter2D(image, -1, kernel)
```

4. Display results

```
plt.figure(figsize=(10, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.title("Original")
```

```
plt.imshow(image, cmap='gray')
```

```
plt.axis('off')
```

```
plt.subplot(1, 2, 2)
```

```
plt.title("Sharpened")
```

```
plt.imshow(sharpened_image, cmap='gray')
```

```
plt.axis('off')
```

```
plt.show()
```